



DAY 7

31 DAYS OF REACT DESIGN PATTERNS

Build a Compound Card

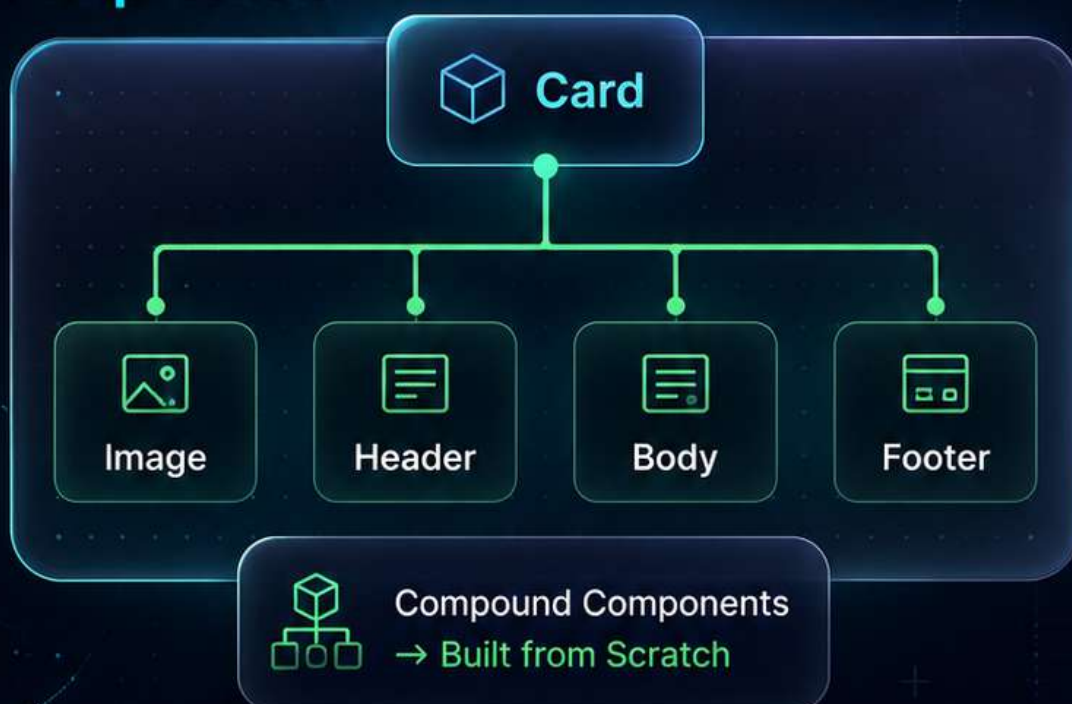
Today's Challenge

Build a reusable Card using composition instead of a giant prop-based API.

```
<Card>
  <Card.Image />
  <Card.Header />
  <Card.Body />
  <Card.Footer />
</Card>
```

No giant prop configuration.
Just **composition**.

```
function Card() {
  return (
    <div className="card">
      {children}
    </div>
  )
}
```





DAY 7

31 DAYS OF REACT DESIGN PATTERNS

The Challenge

Reusable Cards often start simple... but quickly become overloaded with props as requirements grow.



Traditional Prop-Heavy Card



10+ Props to Manage

VS

10+ Props



VS

Composition



Compound Component

```
<Card>
  <Card.Image />
  <Card.Header />
  <Card.Body />
  <Card.Footer />
</Card>
```



Card.Image



Card.Header



Card.Body



Card.Footer



Composable & Flexible API



The goal isn't to add more props.
It's to build reusable components through
composition instead of **configuration**.



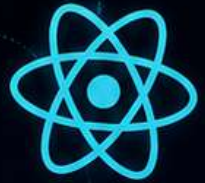


DAY 7

31 DAYS OF REACT DESIGN PATTERNS

What I Needed to Build

Design a reusable Card API that gives developers complete control over layout through composition.



Core Building Blocks



`<Card />`



`<Card.Image />`



`<Card.Header />`



`<Card.Body />`



`<Card.Footer />`



Each piece has one clear responsibility.

Reusable Pieces



Different Layouts



Flexible Composition

Variant A



Image

Header

Body

Footer

Premium Headphones

High quality sound with deep bass and all-day comfort.

\$129.99

Add to Cart



Complete Product Card

Variant B



Header

Body

Getting Started

Everything you need to build faster and smarter.



Minimal Content Card



The same parent component should support **multiple layouts** without adding new props.

`<Card />`





DAY 7

</>

31 DAYS OF REACT DESIGN PATTERNS

How I Structured It

One parent component shares data.

Multiple child components consume only what they need.

{ }

<Card />



ARCHITECTURE FLOW



Parent

Owns the shared data.



Context

Shares data without prop drilling.



Children

Render only what they need.



KEY INSIGHT

The **parent** manages the data.
The **children** focus on rendering.
Context keeps them connected **without passing props** everywhere.

</>



DAY 7

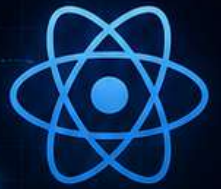
31 DAYS OF REACT DESIGN PATTERNS

How I Built It

Building a Compound Component is a step-by-step process—start with shared data, then compose reusable pieces.

</>

{ }



<Card />

1



Context

Create Shared Context

Store all shared Card data in one place so every related component can access it.

```
1 const CardContext =  
  createContext();
```



Context

2



Parent

Build the Parent

The 'Card' component becomes the provider and wraps every child component.

```
1 <Card>  
2   {children}  
2 </Card>
```



Card



Parent

3

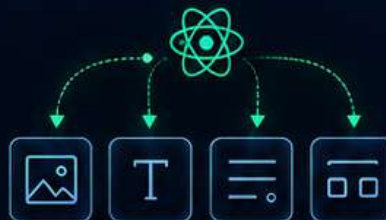


Children

Build Compound Children

Each child consumes only the shared data it needs.

```
1 Card.Image  
2 Card.Header  
3 Card.Body  
4 Card.Footer
```



Children

4



Composition

Compose the UI

```
1 <Card>  
2   <Card.Image />  
3   <Card.Header />  
4   <Card.Body />  
5   <Card.Footer />  
6 </Card>
```

✓ Consumer controls the layout



Card Header

This is the card body content.
It can be anything you want.

Card Footer

o o o

Composition



KEY INSIGHT

The parent provides **shared data**.

The **children** stay focused.

The **consumer decides** the final composition.

</>



DAY 7

31 DAYS OF REACT DESIGN PATTERNS

The Part That Makes It Work

The magic happens when related components become static properties of the parent, creating one clean and intuitive API.



Attach child components to the parent

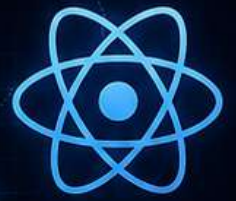
Card.js

```
1 Card.Image = CardImage;  
2 Card.Header = CardHeader;  
3 Card.Body = CardBody;  
4 Card.Footer = CardFooter;
```

Creates a Composable API

App.jsx

```
1 <Card>  
2   <Card.Image />  
3   <Card.Header />  
4   <Card.Body />  
5   <Card.Footer />  
6 </Card>
```



</>

{ }

<Card />



Card



Image



Header



Body



Footer



Composable



Readable



Reusable



Flexible

KEY INSIGHT

Instead of exposing many unrelated components, expose **one parent** with related child components. The API becomes **cleaner**, more discoverable, and **easier to extend**.



</>



DAY 7

31 DAYS OF REACT DESIGN PATTERNS

Where This Pattern Shines

Compound Components are ideal when several UI elements naturally belong together and need shared behavior.



KEY INSIGHT

Whenever **multiple UI elements** work together as one feature, Compound Components provide a **cleaner** and more **flexible API** than adding **endless props**.



When NOT to Use It

Compound Components are powerful—
but they're not the right solution for every component.



Keep It Simple

Simple Components

```
<Button  
  variant="primary"  
  size="lg"  
/>  
  
<Badge color="success" />  
  
<Avatar src={user.image} />
```



Few Props



Easy to Understand



Simple API



Props are enough.



Ask Yourself

- ✓ Does it have multiple related parts?
- ✓ Do child components share data?
- ✓ Does layout need to be customizable?
- ✓ Will more props make the API harder to use?

If most answers are
"Yes" → Compound
Components are a
great fit.



Use Compound Components

Complex Components

```
<Tabs>  
  <Tabs.List />  
  <Tabs.Trigger />  
  <Tabs.Content />  
</Tabs>
```

```
<Modal>  
  <Modal.Header />  
  <Modal.Body />  
  <Modal.Footer />  
</Modal>
```

✓ Shared State

✓ Multiple Parts

✓ Flexible Layout



Composition makes sense here.



KEY INSIGHT

Choose the **simplest solution** that solves the problem.
Use Compound Components only when multiple
related elements need to work together through **shared state**.



DAY 7

31 DAYS OF REACT DESIGN PATTERNS

Real-World Examples

Once you recognize this pattern, you'll start seeing it across modern React UI libraries and design systems.

</>

{ }

<Tabs />



1 Tabs

Overview

Features

Pricing



Overview

This is the overview content of the product.

```
<Tabs>
  <Tabs.List />
  <Tabs.Trigger />
  <Tabs.Content />
</Tabs>
```



✓ Shared State

2 Accordion

What is Compound Components?

A pattern that lets related components work together through a shared parent.

Why use Context?

Best Practices

```
<Accordion>
  <Accordion.Item />
  <Accordion.Trigger />
  <Accordion.Content />
</Accordion>
```



✓ Composable Sections

3 Product Card



Wireless Headphones

Premium sound. All-day comfort.

\$149

🛒 Add to Cart

```
<ProductCard>
  <ProductCard.Image />
  <ProductCard.Header />
  <ProductCard.Body />
  <ProductCard.Footer />
</ProductCard>
```



✓ Flexible Layout

🔄 Reusable

Context

🌐 Shared Data

🧩 Composable

📈 Scalable

Tabs

Accordion

Product Card

KEY INSIGHT

Different interfaces. **One design principle.**

Compound Components make complex UI easier to **organize**, **extend**, and **reuse**.

</>



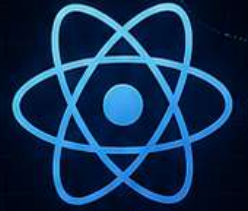
DAY 7

31 DAYS OF REACT DESIGN PATTERNS

</>

{ }

<Card />



Key Takeaways

Compound Components help you design React APIs that are flexible, reusable, and enjoyable to work with.



Compose, Don't Configure

01

Avoid endless props by breaking large components into smaller related pieces.



Share State with Context

02

Keep related components connected without prop drilling.



Build Flexible APIs

03

Consumers decide the layout while the parent manages shared logic.



Improve Developer Experience

04

Readable, scalable, reusable components are easier to maintain and extend.



Card



Image



Header



Body



Footer

One Parent • Multiple Children • One Clean API



Great React components aren't the ones with the most props—they're the ones that make **composition** feel natural.



@mtayyabrawan

Follow for more React Design Patterns

Frontend Developer • React Enthusiast